VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

Danish Kodavanti (1BM24CS086)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Sep-2024 to Jan-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Machine Learning (23CS6PCMAL)" carried out by **Danish Kodavanti (1BM24CS086),** who is bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

Saritha A. N Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

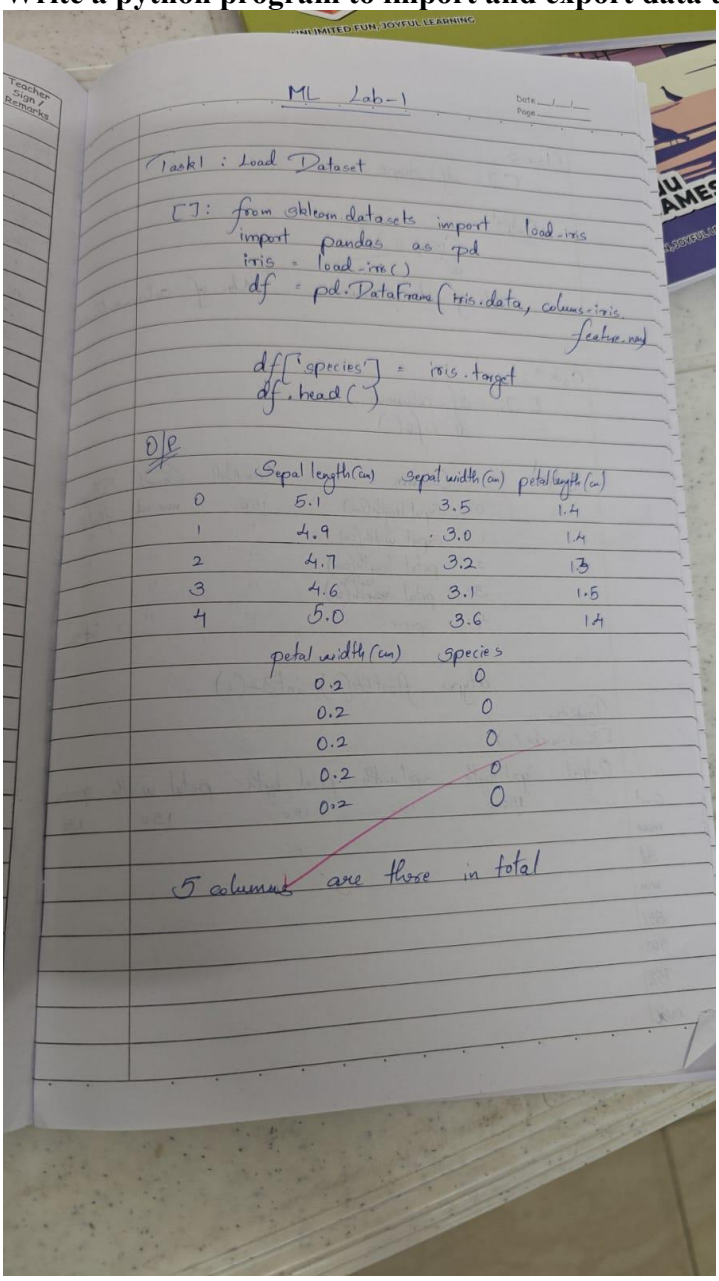
Index

Sl. No.	Date	Experiment Title	Page No.
1	21-2-2025	Write a python program to import and export data using Pandas library functions	4-10
2	3-3-2025	Demonstrate various data pre-processing techniques for a given dataset	11-13
3	10-3-2025	Implement Linear and Multi-Linear Regression algorithm using appropriate dataset	14-16
4	17-3-2025	Build Logistic Regression Model for a given dataset	17-19
5	24-3-2025	Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.	20-23
6	7-4-2025	Build KNN Classification model for a given dataset.	24-25
7	21-4-2025	Build Support vector machine model for a given dataset	26-28
8	5-5-2025	Implement Random forest ensemble method on a given dataset.	29-31
9	5-5-2025	Implement Boosting ensemble method on a given dataset.	32-33
10	12-5-2025	Build k-Means algorithm to cluster a set of data stored in a .CSV file.	34-36
11	12-5-2025	Implement Dimensionality reduction using Principal Component Analysis (PCA) method.	37-41

Github Link: https://github.com/Danishkodavanti/ML_Lab

Program 1

Write a python program to import and export data using Pandas library functions



ML Lab-1

Task1 : Load Dataset

[7]:
```
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)

df['species'] = iris.target
df.head()
```

O/P

	Sepal length(cm)	Sepal width(cm)	petal length(cm)	petal width(cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

5 columns are there in total

Date __/__/__
Page _____

Task 2:

[ ]: df.shape

Output

(150, 5)

No. of entries/rows → No. of columns/features

Task 3:

[ ]: df.columns
df.info()

Output

	Column	Non-Null Count	Dtype
0	Sepal length(cm)	150 non-null	float64
1	Sepal width(cm)	" "	"
2	petal length(cm)	" "	"
3	petal ~~length~~ width(cm)	" "	"
4	species	" "	int32

dtypes: float64(4), int32(1)

Task 4:

[ ]: describe()

Output

	Sepal length	sepal width	petal length	petal width	species
Count	150.0	150	150	150	150
mean					
std					
min					
25%					
50%					
75%					
max					

Date __/__/__
Page _____

7.9 [max, sepal length(cm)] has largest value in feature, its sepal length(cm)

Take 5:

[ ]: df['species'].value_counts()

Output

Species	
0	50
1	50
2	50

Yes all species are balanced //

Task 6:

[ ]: import matplotlib.pyplot as plt
plt.scatter(df['sepal length(cm)'], df['petal length(cm)'])

plt.xlabel("Sepal length")
plt.ylabel("Petal length")
plt.show()

Ya groups are forming

tight group at lower petal length → species 0

Other 2 points form separate clusters at higher petal lengths → species 1 & 2

Date __/__/__
Page ____

Task 7:

```
df[df['species'] == 0].head()
df[df['species'] == 0].head()
```

Species 0
Smaller petal length & width

Species 1
larger petal length & width

Task 8:

```
df.corr()
```

Strong relationships

Petal length & width → 0.96 ≈ +1

other

petal length & species → 0.949
petal length & species → 0.956
sepal length & Petal length → 0.87
sepal length & Petal width → 0.817

Task 9:

```
df.sort_values(by='petal length(cm)').head()
```

Rows are ordered by smallest petal length, all showing species = 0. Only row order changes, species labels remain the same.

Code:

```
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target
df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
df.shape
```

```
(150, 5)
```

```
df.columns
```

```
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
       'petal width (cm)', 'species'],
      dtype='object')
```

```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   species            150 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

```python
df.describe()
```

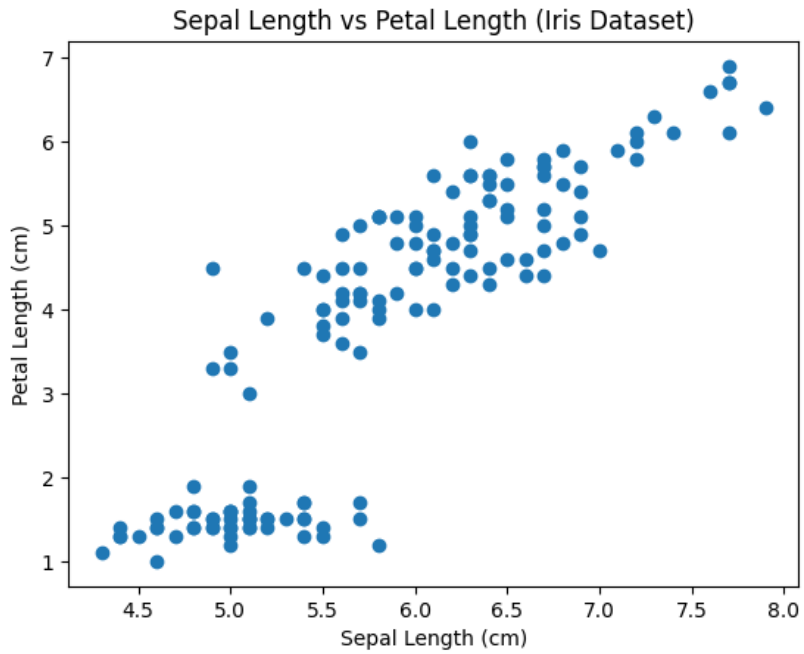
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

```python
df['species'].value_counts()
```

species	count
0	50
1	50
2	50

dtype: int64

```python
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load the Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

# Scatter plot: Sepal Length vs Petal Length
plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Petal Length (cm)")
plt.title("Sepal Length vs Petal Length (Iris Dataset)")
plt.show()
```




```
df[df['species'] == 0].head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
df[df['species'] == 1].head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
50	7.0	3.2	4.7	1.4	1
51	6.4	3.2	4.5	1.5	1
52	6.9	3.1	4.9	1.5	1
53	5.5	2.3	4.0	1.3	1
54	6.5	2.8	4.6	1.5	1

```python
df.corr()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
sepal length (cm)	1.000000	-0.117570	0.871754	0.817941	0.782561
sepal width (cm)	-0.117570	1.000000	-0.428440	-0.366126	-0.426658
petal length (cm)	0.871754	-0.428440	1.000000	0.962865	0.949035
petal width (cm)	0.817941	-0.366126	0.962865	1.000000	0.956547
species	0.782561	-0.426658	0.949035	0.956547	1.000000

```python
df.sort_values(by='petal length (cm)').head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
22	4.6	3.6	1.0	0.2	0
13	4.3	3.0	1.1	0.1	0
14	5.8	4.0	1.2	0.2	0
35	5.0	3.2	1.2	0.2	0
16	5.4	3.9	1.3	0.4	0

Program 2

Demonstrate various data pre-processing techniques for a given dataset

Date __/__/__
Page _____

ML Lab2

House ~~Predicti~~ Price Prediction

Step 0

Import all the required libraries like numpy, pandas, matploltlib.pyplot, Seaborn, Sklearn and import datasets

Step1

Problem Type: Supervised learning - Regression
Objective: Predicting median house value based on housing features by using RMSE, MAE, R^2

Step2

Get the Data and display basic info, shape, head

Step3

discover data

Visualize the Data

Step4

Distribution of House Prices
Income vs House Price
Correlation Heatmap
Location-based Visualization

Plot graphs for each ~~co~~ topic and analyse the given data set

Step 5 Prepare the Data

Train - Test Split

train 80% test 20%

Consider a random - state = 42

Feature Scaling

Step 6

Select and Train base model

Linear regression Descision Tree Regressor Random Forest Regressor

Step 7

Fine - tuning of Random Forest

Step 8 Model Comparison

Random forest is best

Reasons

→ Non - linearity
→ Feature interactions
→ Reduced overfitting vs Single tree
→ Robustness to noise and outliers.

Tuning adjusts hyperparameters
→ n - estimators
→ max depth
→ max samples split
→ min samples leaf
→ max features

additional data to improve predictions

Better location features
→ Distance to ocean
→ city center, etc
Economic factors
Interest rate, etc
Prop features
→ Lot size
→ prop rate, etc.

Code:

```
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target
df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
df.shape
```

```
(150, 5)
```

```
df.columns
```

```
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
       'petal width (cm)', 'species'],
      dtype='object')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   species            150 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

```
df.describe()
```

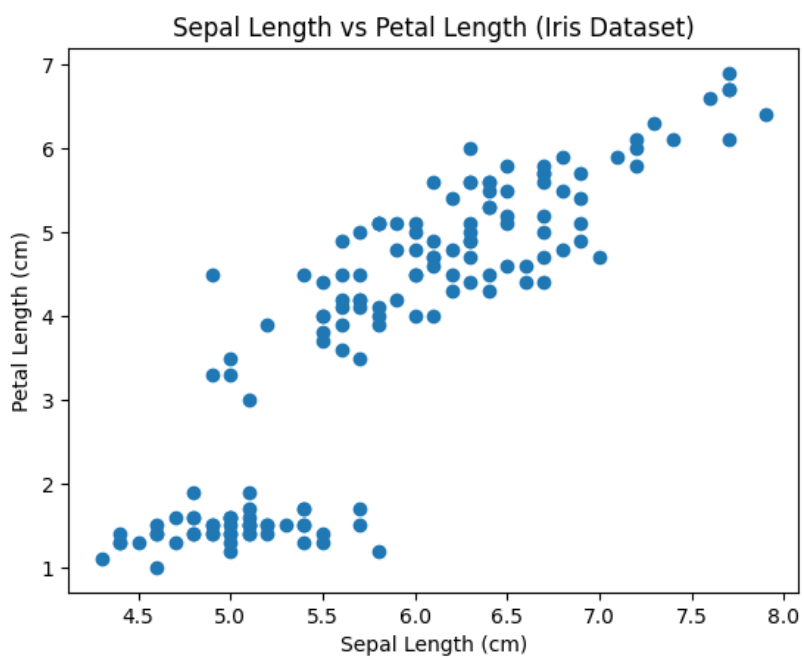
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

```python
df['species'].value_counts()
```

species	count
0	50
1	50
2	50

dtype: int64

```python
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load the Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

# Scatter plot: Sepal Length vs Petal Length
plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Petal Length (cm)")
plt.title("Sepal Length vs Petal Length (Iris Dataset)")
plt.show()
```



```
df[df['species'] == 0].head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
df[df['species'] == 1].head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
50	7.0	3.2	4.7	1.4	1
51	6.4	3.2	4.5	1.5	1
52	6.9	3.1	4.9	1.5	1
53	5.5	2.3	4.0	1.3	1
54	6.5	2.8	4.6	1.5	1

```
df.corr()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
sepal length (cm)	1.000000	-0.117570	0.871754	0.817941	0.782561
sepal width (cm)	-0.117570	1.000000	-0.428440	-0.366126	-0.426658
petal length (cm)	0.871754	-0.428440	1.000000	0.962865	0.949035
petal width (cm)	0.817941	-0.366126	0.962865	1.000000	0.956547
species	0.782561	-0.426658	0.949035	0.956547	1.000000

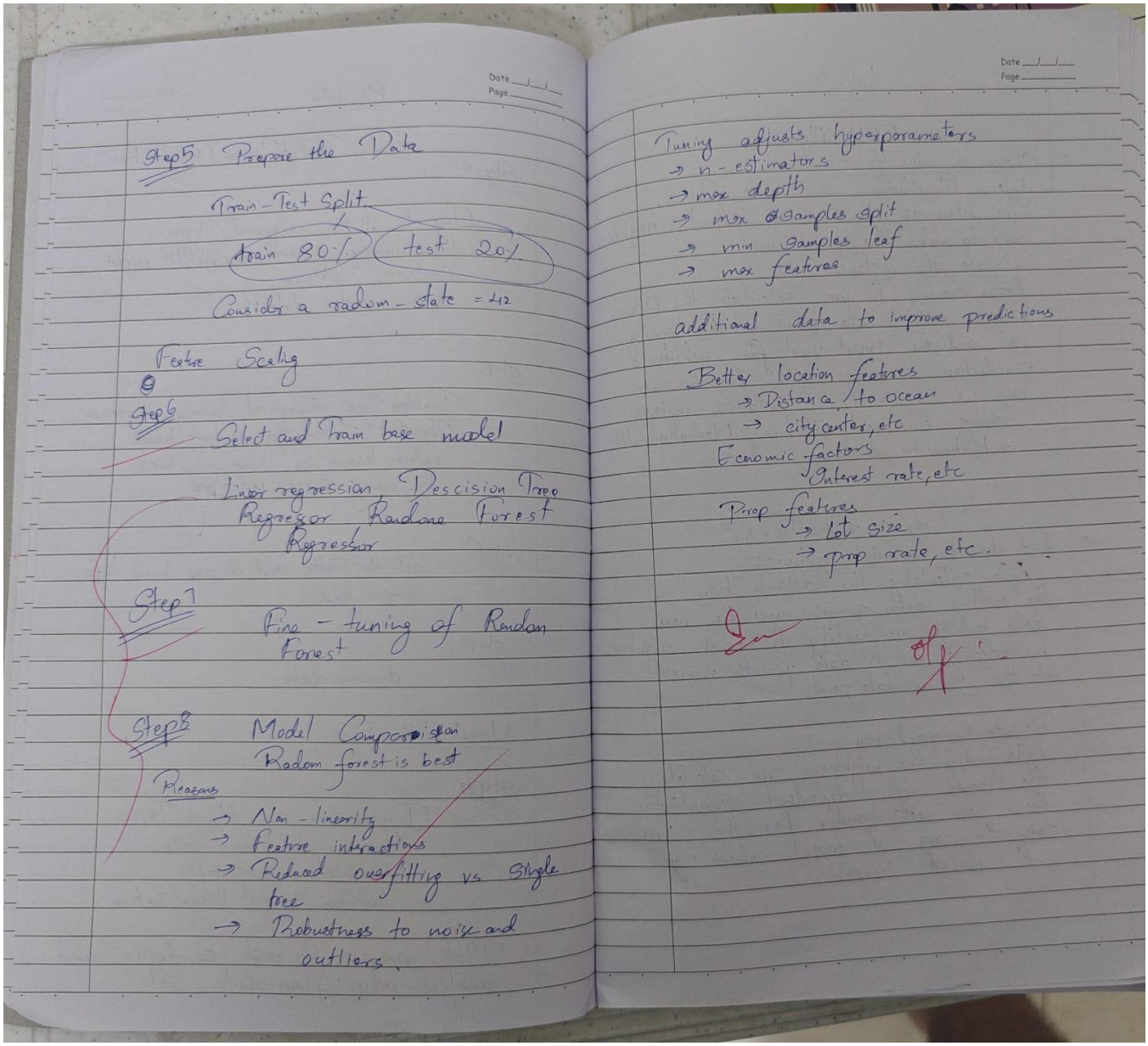
```
df.sort_values(by='petal length (cm)').head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
22	4.6	3.6	1.0	0.2	0
13	4.3	3.0	1.1	0.1	0
14	5.8	4.0	1.2	0.2	0
35	5.0	3.2	1.2	0.2	0
16	5.4	3.9	1.3	0.4	0

Program 3

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.

Screenshot



Step5 Prepare the Data

Train-Test Split

train 80% test 20%

Consider a random-state = 42

Feature Scaling

Step6

Select and Train base model

Linear regression Descision Tree Regressor Random Forest Regressor

Step7

Fine-tuning of Random Forest

Step8 Model Comparision

Random forest is best

Reasons

→ Non-linearity
→ Feature interactions
→ Reduced overfitting vs single tree
→ Robustness to noise and outliers.

Tuning adjusts hyperparameters

→ n-estimators
→ max depth
→ max samples split
→ min samples leaf
→ max features

additional data to improve predictions

Better location features

→ Distance to ocean
→ city center, etc

Economic factors

Interest rate, etc

Prop features

→ lot size
→ prop rate, etc.

Date __/__/__
Page _____

Decision Tree Regressor (CART)

```
reg = DecisionTreeRegressor()
reg.fit(X_train, y_train)
y_pred = reg.predict(X_test)

print(mean_squared_error(y_test, y_pred))
print(mean_absolute_error(y_test, y_pred))
print(r2_score(y_test, y_pred))
```

O/P

MSE: 4593.359
MAE: 52.0113
R2 Score: 0.1330

Code:

```python
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = DecisionTreeClassifier(criterion='entropy', random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

new_sample = [[5.1, 3.5, 1.4, 0.2]]

prediction = model.predict(new_sample)

print("Predicted class:", prediction[0])
print("Predicted species:", iris.target_names[prediction][0])
```

```
Accuracy: 1.0
Predicted class: 0
Predicted species: setosa
```

Program 4

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset

Date __/__/__
Page _____

Lab - 5 logistic Regression

Binary Classification - HR DataSet

Aim

To predict whether an employee will leave or stay in the company using logistic Regression (Binary Classification

Contains 14,999 employee records.

Feature

Satisfaction level	0 to 1
last - evaluation	last performance evaluation score
number - project	No. of projects assigned.
avg - montly - ~~years~~hrs	Avg montly working hrs
time - spent	Years spent in Company
Salary - hike	Expected hike based on performance
Promotion - 5years	promoted in last 5years

Date __/__/__
Page ____

Objective
Perform EDA
Plot bar charts
Binary logistic Regression

Logistic Regression uses the Sigmoid func to map the output to a prob b/w 0 to 1

Multiclass Classification

Zoo Dataset

Aim
To predict type of animal (7 categories) based on its physical features using logistic Regression

101 animal records with 16 boolean/numeric feat a 1 target Column

Feat

hair, feathers, eggs — 0 or 1
milk, airborne, aquatic — 0 or 1

legs — No. of legs
tail, domestic, catsize — More phy traits

Class_type — target - 1 = Mam
2 = Bird, ... etc

Date __/__/__
Page ____

Binary Classification - Student Pass/Fail

1. $P(y=1) = \frac{1}{1+e^{-(a_0+a_1x)}}$

$\boxed{a_0 = -5,\ a_1 = 0.8}$

$P(pass) = \frac{1}{1+e^{-(-5+0.8x)}}$ //

2. Prob for 7 hrs Study

$z = a_0 + a_1 x = 0.6$

$P(pass) = \frac{1}{1+e^{-z}} = \frac{1}{1+0.5488}$

$= 0.6457$

64.57% //

3. SoftMax func for $z = [2, 1, 0]$

$P(z_i) = \frac{e^z}{e^{z_1}+e^{z_2}+e^{z_3}}$

$e^2 = 7.3$
$e^1 = 2.7$
$e^0 = 1.$

Date __/__/__
Page _____

Sum = 7.38 + 2.7 + 1.0 = 11

2

Class 1	7.3 / 11.1	66.5 %
2	2.7 / 11.1	24.47 %
3	1.0 / 11.1	9. %

Total = 1.0

Code:

```
X = df.drop('MedHouseVal', axis=1)
y = df['MedHouseVal']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
lin_reg = LinearRegression()
lin_reg.fit(X_train_scaled, y_train)

y_pred_lr = lin_reg.predict(X_test_scaled)

rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred_lr))
mae_lr = mean_absolute_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)

rmse_lr, mae_lr, r2_lr
```

```
(np.float64(0.7449540928043491), 0.5330139776645934, 0.5765012224420747)
```

```
dt = DecisionTreeRegressor(random_state=42)
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)
rmse_dt = np.sqrt(mean_squared_error(y_test, y_pred_dt))

rmse_dt
```

```
np.float64(0.7138377373947808)
```

```
rf = RandomForestRegressor(random_state=42)
rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))

rmse_rf
```

```
np.float64(0.5128644706054181)
```

```
param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5]
}

grid_search = GridSearchCV(
    rf,
    param_grid,
    cv=3,
    scoring='neg_mean_squared_error'
)
```

```python
results = pd.DataFrame({
    'Model': ['Linear Regression',
              'Decision Tree',
              'Random Forest',
              'Tuned Random Forest'],
    'RMSE': [rmse_lr, rmse_dt, rmse_rf, rmse_best]
})

results
```

	Model	RMSE
0	Linear Regression	0.744954
1	Decision Tree	0.713838
2	Random Forest	0.512864
3	Tuned Random Forest	0.513263

Program 5

Build Logistic Regression Model for a given dataset

Date __/__/__
Page _____

Objective
Perform EDA
Plot barcharts
Binary logistic Regression

Logistic Regression uses the Sigmoid func to map the output to a prob b/w 0 to 1

Multiclass Classification

Zoo Dataset

Aim
To predict type of animal (7 categories) based on its physical features using logistic Regression

101 animal records with 16 boolean/numeric feat a 1 target Column

Feat

hair, feathers, eggs — 0 or 1
milk, airborne, aquatic — 0 or 1

legs — No. of legs
tail, domestic, catsize — More phy traits

Class_type — target - 1 = Mammal
2 = Bird, ... etc

Date __/__/__
Page _____

Binary Classification - Student Pass/Fail

1. $P(y=1) = \frac{1}{1+e^{-(a_0+a_1x)}}$

$\boxed{a_0 = -5,\ a_1 = 0.8}$

$P(pass) = \frac{1}{1+e^{-(-5+0.8x)}}$ //

2. Prob for 7 hrs Study

$z = a_0 + a_1 x = 0.6$

$P(pass) = \frac{1}{1+e^{-z}} = \frac{1}{1+0.5488}$

$= 0.6457$

64.57% //

3. SoftMax func for $z = [2, 1, 0]$

$P(z_i) = \frac{e^z}{e^{z_1}+e^{z_2}+e^{z_3}}$

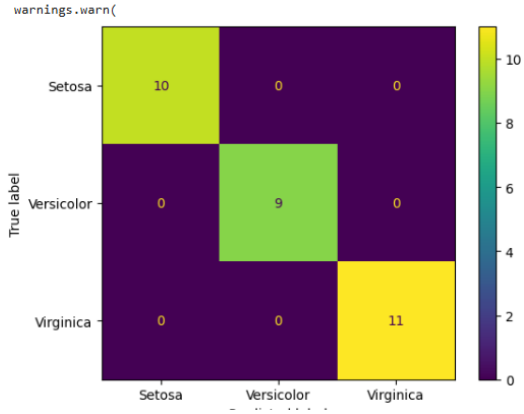
$e^2 = 7.3$
$e^1 = 2.7$
$e^0 = 1.$

Code:

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn import metrics
import matplotlib.pyplot as plt

# Load correct file
iris = pd.read_csv("iris (1).csv")

print(iris.head())

# Features and target
X = iris.drop('species', axis='columns')
y = iris['species']

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Model
model = LogisticRegression(multi_class='multinomial', solver='lbfgs')
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```python
# Confusion Matrix
cm = metrics.confusion_matrix(y_test, y_pred)
disp = metrics.ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Setosa", "Versicolor", "Virginica"]
)
disp.plot()
plt.show()
```

```
   sepal_length  sepal_width  petal_length  petal_width      species
0           5.1          3.5           1.4          0.2  Iris-setosa
1           4.9          3.0           1.4          0.2  Iris-setosa
2           4.7          3.2           1.3          0.2  Iris-setosa
3           4.6          3.1           1.5          0.2  Iris-setosa
4           5.0          3.6           1.4          0.2  Iris-setosa
Accuracy: 1.0
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated i
  warnings.warn(
```



Program 6

Build KNN Classification model for a given dataset

Date __/__/__
Page _____

Objective
Perform EDA
Plot barcharts
Binary logistic Regression

Logistic Regression uses the Sigmoid func to map the output to a prob b/w 0 to 1

Multiclass Classification

Zoo Dataset

Aim
To predict type of animal (7 categories) based on its physical features using logistic Regression

101 animal records with 16 boolean/numeric feat a 1 target Column

Feat

hair, feathers, eggs — 0 or 1
milk, airborne, aquatic — 0 or 1

legs — No. of legs
tail, domestic, catsize — More phy traits

Class_type — target - 1 = Mam
2 = Bird, ... etc

Date __/__/__
Page _____

Binary Classification - Student Pass/Fail

1. $P(y=1) = \frac{1}{1+e^{-(a_0+a_1x)}}$

$\boxed{a_0 = -5,\ a_1 = 0.8}$

$P(pass) = \frac{1}{1+e^{-(-5+0.8x)}}$ //

2. Prob for 7 hrs Study

$z = a_0 + a_1 x = 0.6$

$P(pass) = \frac{1}{1+e^{-z}} = \frac{1}{1+0.5488}$

$= 0.6457$

64.57% //

3. SoftMax func for $z = [2, 1, 0]$

$P(z_i) = \frac{e^z}{e^{z_1}+e^{z_2}+e^{z_3}}$

$e^2 = 7.3$
$e^1 = 2.7$
$e^0 = 1.$

Code:

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import mean_squared_error, r2_score

from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import DecisionTreeRegressor
```

```
from sklearn.datasets import load_iris

data = load_iris()

X = data.data
y = data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
model = DecisionTreeClassifier(criterion='entropy')

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

print(classification_report(y_test, y_pred))
```

```
Accuracy: 1.0
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

```
from sklearn.datasets import load_wine

data = load_wine()

X = data.data
y = data.target
```

```
from sklearn.datasets import load_diabetes

data = load_diabetes()

X = data.data
y = data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
regressor = DecisionTreeRegressor()

regressor.fit(X_train, y_train)

y_pred = regressor.predict(X_test)
```

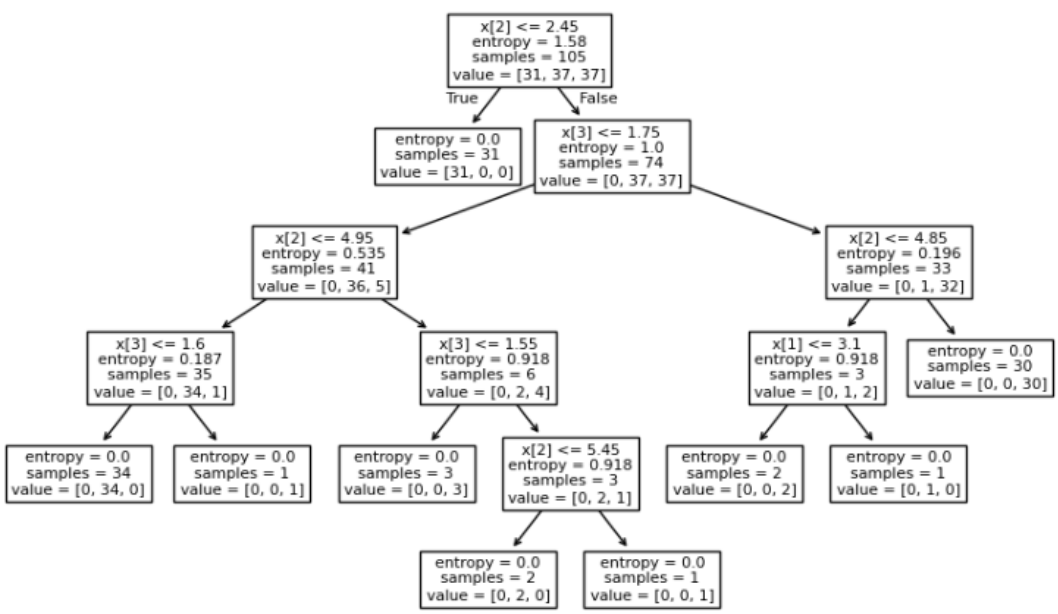
```
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("R2 Score:", r2)
```

```
Mean Squared Error: 5358.458646616541
R2 Score: 0.0073817790735694055
```

```
from sklearn.datasets import fetch_california_housing

data = fetch_california_housing()

X = data.data
y = data.target
```

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
plot_tree(model)
plt.show()
```

...



Program 7

Build Support vector machine model for a given dataset

Steps

→ Import dataset

→ Separate features and target

```
X = data.drop('quality', axis=1)
y = data['quality']

Scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

→ Standardize the data

→ Apply PCA

```
pca = PCA(n_components = 2)
X_pca = pca.fit_transform(X_scaled)
print("Reduced Shape:", X_pca.shape)
```

→ Reduce dimension

→ Visualize transformed data.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from ma
import matplotlib.pyplot as plt
df = pd.read_csv("glass_dataset.csv")

print(df.head())
print(df.info())
X = df.drop(columns=["Type"])
y = df["Type"]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA(n_comp = 2)
X_pca.explaind_variance_ratio

pca_df = pd.DataFrame(data = X_pca,
columns = ["PC1" "PC2"])
pca_df["Target"] = y.values

print(pca_df.head())

plt.figure(figsize = (8,6))
```

Code:

```python
# Import libraries
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler

# Load dataset
data = datasets.load_iris()
X = data.data
y = data.target

# Feature scaling (important for SVM)
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Split dataset (fixed for same results every time)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create SVM model
model = SVC(kernel='linear')

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
```

```python
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)

# Classification Report
cr = classification_report(y_test, y_pred)
print("\nClassification Report:\n", cr)
```

```
Accuracy: 0.9666666666666667

Confusion Matrix:
 [[10  0  0]
 [ 0  8  1]
 [ 0  0 11]]

Classification Report:
               precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      0.89      0.94         9
           2       0.92      1.00      0.96        11

    accuracy                           0.97        30
   macro avg       0.97      0.96      0.97        30
weighted avg       0.97      0.97      0.97        30
```

Program 8

Implement Random forest ensemble method on a given dataset.

Screenshot

Lab - 6

Date __/__/__
Page _____

Random forest

Aim: To implement the Random Forest ensemble method on a given dataset and analyze its performance for classification / regression.

Introduction

It uses bagging (Bootstrap Aggregation)
Each tree is trained on a random subset

Implementation

data = pd.read_csv("data.csv")

x = data.drop("target", axis=1)

y = data["target"]

Split

20% test 80% train

model = RandomForestClassifier (n_estimators = 100, random_state=42)

Date __/__/__
Page _____

accuracy = accuracy_score (y_test, y_pre
print ("Accuracy:", accuracy)

O/P

98.228 %

Parameters Used

n_estimators: No. of trees
max_depth: Max depth of trees
random_state: For reproducibility
criterion: Splitting method

Conclusion

Random Forest is a powerful and reliable machine learning algorithm.

Code:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(n_estimators=100, random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```
Accuracy: 1.0
Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

Program 9

Implement Boosting ensemble method on a given dataset.

Date __/__/__
Page _____

Lab-7

Boosting

Load Dataset

```
from google colab import files
uploaded = files.upload()

data = pd.read_csv(list(uploaded.keys
                        ())[0])

data.head()
```

Train - Test Split

```
X_train, X_test, y_train, y_test = train_test_
                                   split(
    X, y, test_size = 0.2, random_state
                           = 42)
```

Apply Boosting (Ada Boost)

```
base_model = DecisionTreeClassifier
                 (max_depth=1)

model = AdaBoostClassifier
    (
      estimator = base_model,
      n_estimators = 50,
      learning_rate = 1.0,
      random_state = 42
    )

model.fit(X_train, y_train)
```

Date __/__/__
Page _____

Prediction & Accuracy

```
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score
                 (y_test, y_pred))
```

Confusion Matrix & Report

```
print("Confusion Matrix:\n", confusion_
                 matrix
                 (y_test, y_pred))

print("\n Classification Report:\n",
                 classification_report
                 (y_test, y_pred))
```

Feature Importance Graph

```
importance = model.feature_importances_
plt.figure()
plt.barh(range(len(importance)), importance)
```

Confusion Matrix

```
[[40  3]
 [1  70]]
```

Classification Report:

	precision	recall	f1-score
0	0.98	0.93	0.95
1	0.96	0.99	0.97

Code:

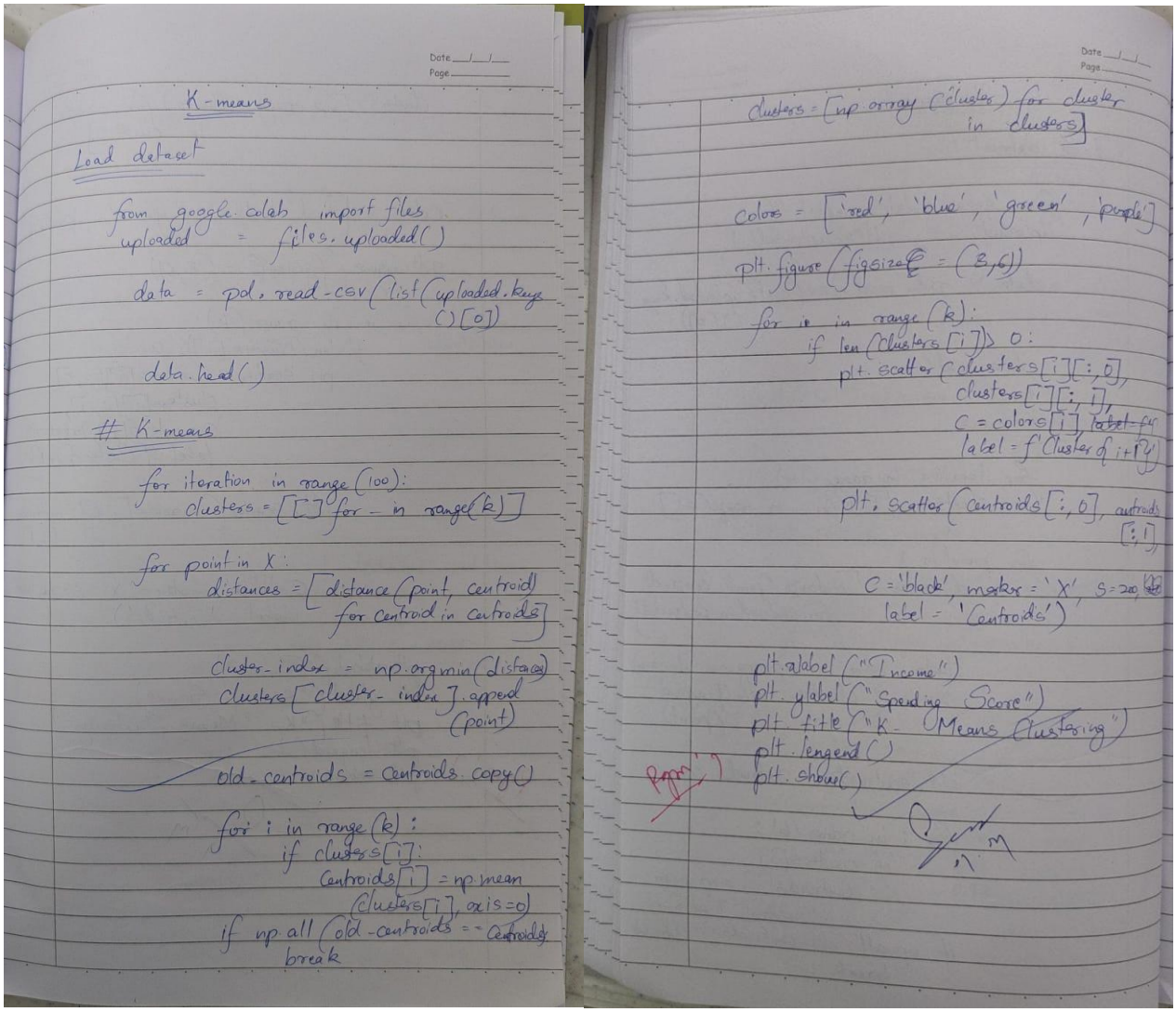
```python
# Import libraries
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load Wine dataset
data = load_wine()
X = data.data
y = data.target

# Split dataset into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create AdaBoost model
model = AdaBoostClassifier(
    n_estimators=50,      # number of weak learners
    learning_rate=1.0,    # contribution of each classifier
    random_state=42
)

# Train model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```python
# Evaluate model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.9444444444444444

Classification Report:
               precision    recall  f1-score   support

           0       0.93      1.00      0.97        14
           1       0.93      0.93      0.93        14
           2       1.00      0.88      0.93         8

    accuracy                           0.94        36
   macro avg       0.95      0.93      0.94        36
weighted avg       0.95      0.94      0.94        36
```

Program 10

Build k-Means algorithm to cluster a set of data stored in a .CSV file.



K-means

Load dataset

```
from google.colab import files
uploaded = files.uploaded()

data = pd.read_csv(list(uploaded.keys()[0])

data.head()

# K-means

for iteration in range(100):
    clusters = [[] for _ in range(k)]

    for point in X:
        distances = [distance(point, centroid)
                     for centroid in centroids]

        cluster_index = np.argmin(distances)
        clusters[cluster_index].append(point)

    old_centroids = centroids.copy()

    for i in range(k):
        if clusters[i]:
            centroids[i] = np.mean(clusters[i], axis=0)
    if np.all(old_centroids == centroids):
        break

clusters = [np.array(cluster) for cluster in clusters]

colors = ['red', 'blue', 'green', 'purple']

plt.figure(figsize = (8,6))

for i in range(k):
    if len(clusters[i]) > 0:
        plt.scatter(clusters[i][:,0],
                    clusters[i][:,1],
                    c = colors[i] label=f'
                    label = f'Cluster {i+1}')

plt.scatter(centroids[:,0], centroids[:,1],
            c='black', marker='X', s=200,
            label='Centroids')

plt.xlabel("Income")
plt.ylabel("Spending Score")
plt.title("K-Means Clustering")
plt.legend()
plt.show()
```

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

```
data = pd.read_csv("/content/data.csv")
print(data.head())
```

```python
X = data.iloc[:, :].values   # or choose specific columns
```

```python
k = 3   # you can change this
```

```python
kmeans = KMeans(n_clusters=k, random_state=0)
kmeans.fit(X)
```

```python
labels = kmeans.labels_
print(labels)
```

```python
centroids = kmeans.cluster_centers_
print(centroids)
```

```python
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.scatter(centroids[:, 0], centroids[:, 1], s=200, c='red')
plt.title("K-Means Clustering")
plt.show()
```

Program 11

Implement Dimensionality reduction using Principle Component Analysis (PCA) method.

Date ?
Page

Principal Component Analysis

Aim

To perform Dimensionality Reduction using the Principal Component Analysis (PCA) technique on a dataset

Dataset

We use the ~~Wine Quality Dataset~~ Glass Identification

Contains features like:

acidity
sugar
pH
alcohol
Sulphates
etc.

Theory

PCA is a statistical technique used to

→ Reduce the no. of features
→ Preserve max variance
→ transform data into a new co ordinate system

Algo ?

Key Ideas

→ Converts correlated variables
→ New variables are called Principal Components
→ PC1 captures max variance, PC2 next, and so on.

Date __/__/__
Page ____

Steps

→ Import dataset

→ Separate features and target

```
X = data.drop('quality', axis=1)
y = data['quality']

Scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

→ Standardize the data

→ Apply PCA

```
pca = PCA(n_components = 2)
X_pca = pca.fit_transform(X_scaled)
print("Reduced Shape:", X_pca.shape)
```

→ Reduce dimension

→ Visualize transformed data.

Date __/__/__
Page ____

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

df = pd.read_csv("glass_dataset.csv")

print(df.head())
print(df.info())

X = df.drop(columns=["Type"])
y = df["Type"]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA(n_comp = 2)
X_pca.explaind_variance_ratio

pca_df = pd.DataFrame(data = X_pca,
columns = ["PC1" "PC2"])
pca_df["Target"] = y.values

print(pca_df.head())

plt.figure(figsize = (8,6))
```

Date __/__/__
Page ____

```
for label in np.unique(y):
    plt.scatter(
        pca_df[pca_df["Target"] == label["PC1"]]
        pca_df[pca_df["Target"] == label]["PC2"]
        label = label
    )
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA on glass Dataset")
plt.legend()
plt.show
```

O/P

Explained Variance Ratio [0.45 0.23]

16.04.2026

Code:

PCA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

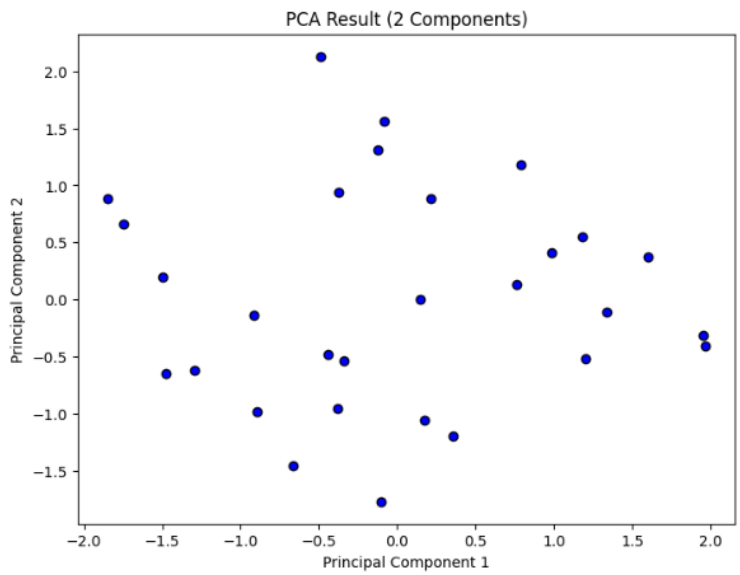
```
# Load dataset
data = pd.read_csv("data.csv")
print(data.head())
```

```
    x   y
0  30  49
1  31  27
2  31  46
3   1  65
4  81  76
```

```
# Standardize the data
scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)
```

```
# Apply PCA
pca = PCA(n_components=2)  # Reduce to 2 components
pca_result = pca.fit_transform(scaled_data)
```

```python
# Explained variance ratio
print(f"Explained Variance Ratio: {pca.explained_variance_ratio_}")
print(f"Total Variance Explained by 2 Components: {np.sum(pca.explained_variance_ratio_)}")
```

```
Explained Variance Ratio: [0.56506606 0.43493394]
Total Variance Explained by 2 Components: 1.0
```

```python
# Scatter plot of the 2D PCA result
plt.figure(figsize=(8, 6))
plt.scatter(pca_result[:, 0], pca_result[:, 1], c='blue', edgecolors='black')
plt.title('PCA Result (2 Components)')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()
```